

Introduction to Cryptography

Lei Xu

lxu12@kent.edu

Computer Science Department

Kent State University

September 17, 2026

Quiz

- Which of the following is NOT a drawback of the one-time pad?
 - A given key can only be used to encrypt one message
 - The key is as long as the message
 - The key must be chosen uniformly
 - Encryption and decryption require computationally expensive operations

In this class

- *Generating a truly random key*
 - Algorithms assume uniform bits; real systems must produce them
 - Collect entropy, then *smooth* it into uniform bits; min-entropy measures what we have to work with
- *Implementing the one-time pad*
 - Gen, Enc, Dec, and what goes wrong when a key is reused
 - Its drawbacks are *inherent*: perfect secrecy forces keys at least as long as messages
- *Toward computational secrecy*
 - Perfect secrecy is stronger than we need — relax it: allow tiny failure probability, restrict to efficient attackers
 - First step: restate perfect secrecy as a randomized *indistinguishability experiment*

Generating a truly random key

Where do random keys come from?

- When describing algorithms, we assume access to uniformly distributed bits/bytes
- Where do these actually come from?
- *Random-number generation*

Random-number generation in practice

- Precise details depend on the system
 - Linux or unix: `/dev/random` or `/dev/urandom`
 - **Do not use `rand()` or `java.util.Random`**
 - Use crypto libraries instead

Random-number generation: two steps

- Two steps:
 - Continually collect a “pool” of high-entropy (i.e., “unpredictable”) data
 - When random bits are requested, process this data to generate a sequence of uniform, independent bits/bytes
 - * May “block” if insufficient entropy available

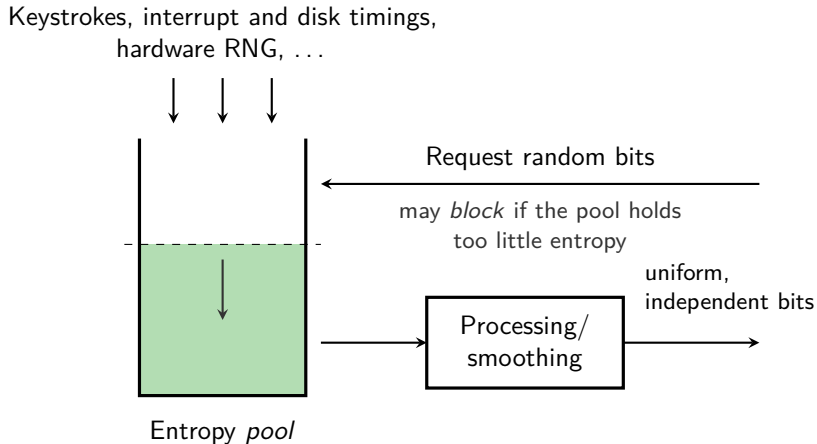
Step 1: Collecting entropy

- Collect a “pool” of high-entropy data
- Must ultimately come from some physical process (since computation is deterministic)
 - External inputs
 - * Keystroke/mouse movements
 - * Delays between network events
 - * Hard-disk access times
 - * Other external sources
 - Hardware random-number generation (e.g., Intel)

Min-entropy

- I.e., “guessing entropy”
- The *min-entropy* of a random variable X is defined as $H_\infty(X) = -\log_2 \max_x \{\Pr[X = x]\}$ (in bits)
 - Min-entropy is determined by the most likely outcome of X
- If X ranges over n -bit strings, then $H_\infty(X) \leq n$
 - Equality iff X has uniform distribution

Putting the two steps together



Entropy accumulates in the pool; each request drains it, so the level falls as bits are handed out.

Step 2: Smoothing

- Need to eliminate both *bias* and *dependencies*
- von Neumann technique for eliminating bias:
 - Collect two bits per output bit
 - * $01 \rightarrow 0$
 - * $10 \rightarrow 1$
 - * $00, 11 \rightarrow \text{skip}$
 - Note that this assumes *independence* (as well as constant bias)

Why the von Neumann method works?

- Assuming that the bits are generated independently and with constant probability
- But it does not require 1 and 0 to be equally likely

Smoothing: randomness extraction

- Can use *randomness extraction*
- Unkeyed extraction is possible for some input distributions; impossible for others
- Keyed extraction possible for all distributions
 - Extracted randomness is less than the input min-entropy
 - Where does the key come from?
- In practice, computational extraction is used

Random extraction is a deep topic

- See, e.g., https://en.wikipedia.org/wiki/Randomness_extractor

Implementing the one-time pad

Key generation

- Read desired number of bytes from `/dev/urandom`
- Output the result to a file

Encryption

- Plaintext = sequence of ASCII characters
- Key = sequence of hex digits
- Read them; XOR them to get the ciphertext

Decryption

- Reverse encryption
- Read ciphertext and key; XOR them to recover the message

Pros of one-time pad

- The one-time pad achieves perfect secrecy!
- One-time pad has historically been used in the real world
 - E.g., “red phone” between DC and Moscow
- Not currently used!
 - Why not?

Cons of one-time pad

- Several limitations
 - The key is as long as the message
 - Only secure if each key is used to encrypt a *single* message
 - * (Trivially broken by a known-plaintext attack)
- $\implies$ Parties must share keys of (total) length equal to the (total) length of all the messages they might ever send

Using the same key twice?

- Say $c_1 = k \oplus m_1$ $c_2 = k \oplus m_2$
- Attacker can compute $c_1 \oplus c_2 = (k \oplus m_1) \oplus (k \oplus m_2) = m_1 \oplus m_2$
- This leaks information about m_1, m_2 !

Why the leak matters

- $m_1 \oplus m_2$ is information about m_1, m_2
- Is this significant?
 - No longer perfectly secret!
 - * $m_1 \oplus m_2$ reveals where m_1, m_2 differ
 - * Frequency analysis
 - * Exploiting characteristics of ASCII...

Characteristics of ASCII

Hex	Dec	Char	Hex	Dec	Char	Hex	Dec	Char
0x20	32	Space	0x40	64	@	0x60	96	`
0x21	33	!	0x41	65	A	0x61	97	a
0x22	34	"	0x42	66	B	0x62	98	b
0x23	35	#	0x43	67	C	0x63	99	c
0x24	36	\$	0x44	68	D	0x64	100	d
0x25	37	%	0x45	69	E	0x65	101	e
0x26	38	&	0x46	70	F	0x66	102	f
0x27	39	'	0x47	71	G	0x67	103	g
0x28	40	(	0x48	72	H	0x68	104	h
0x29	41	)	0x49	73	I	0x69	105	i
0x2A	42	*	0x4A	74	J	0x6A	106	j
0x2B	43	+	0x4B	75	K	0x6B	107	k
0x2C	44	,	0x4C	76	L	0x6C	108	l
0x2D	45	-	0x4D	77	M	0x6D	109	m
0x2E	46	.	0x4E	78	N	0x6E	110	n
0x2F	47	/	0x4F	79	O	0x6F	111	o
0x30	48	0	0x50	80	P	0x70	112	p
0x31	49	1	0x51	81	Q	0x71	113	q
0x32	50	2	0x52	82	R	0x72	114	r
0x33	51	3	0x53	83	S	0x73	115	s
0x34	52	4	0x54	84	T	0x74	116	t
0x35	53	5	0x55	85	U	0x75	117	u
0x36	54	6	0x56	86	V	0x76	118	v
0x37	55	7	0x57	87	W	0x77	119	w
0x38	56	8	0x58	88	X	0x78	120	x
0x39	57	9	0x59	89	Y	0x79	121	y
0x3A	58	:	0x5A	90	Z	0x7A	122	z
0x3B	59	;	0x5B	91	[	0x7B	123	{
0x3C	60	<	0x5C	92	\	0x7C	124	
0x3D	61	=	0x5D	93	]	0x7D	125	}
0x3E	62	>	0x5E	94	^	0x7E	126	~
0x3F	63	?	0x5F	95	_	0x7F	127	DEL

Source: <http://benborowiec.com/2011/07/23/better-ascii-table/>

- Letters all begin with 01...
- The space character begins with 00...
- XOR of two letters gives 00...
- XOR of letter and space gives 01...
- Easy to identify XOR of letter and space!

Spotting the space character

m_1	01...	01...	01...	00...	...
m_2	01...	01...	01...	01...	...
$m_1 \oplus m_2$	00...	00...	00...	01010000	...

$$01010000 = \underbrace{00100000}_{\text{space}} \oplus \underbrace{01110000}_{\text{p}}$$

Drawbacks are inherent

- Drawbacks
 - Key as long the message
 - Only secure if each key is used to encrypt *once*
 - Trivially broken by a known-plaintext attack
- These limitations are *inherent* for schemes achieving perfect secrecy

Optimality: theorem and intuition

- Theorem: if $(\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is perfectly secret, then $|\mathcal{K}| \geq |\mathcal{M}|$.
- Intuition:
 - Given any ciphertext, try decrypting under every possible key in $\mathcal{K}$
 - This gives a list of up to $|\mathcal{K}|$ possible messages
 - If $|\mathcal{K}| < |\mathcal{M}|$, some message is not on the list

Optimality: setting up the proof

- Theorem: if $(\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is perfectly secret, then $|\mathcal{K}| \geq |\mathcal{M}|$.
- Recall: the scheme is *perfectly secret* if for every distribution over $\mathcal{M}$, every $m \in \mathcal{M}$, and every ciphertext $c \in \mathcal{C}$ with $\Pr[C = c] > 0$,

$$\Pr[M = m \mid C = c] = \Pr[M = m].$$

- Proof:
 - Assume $|\mathcal{K}| < |\mathcal{M}|$
 - To contradict perfect secrecy, negate the definition: it suffices to exhibit *one* distribution on $\mathcal{M}$, *one* message m , and *one* ciphertext c with $\Pr[M = m \mid C = c] \neq \Pr[M = m]$

Optimality: completing the proof

- Proof, continued
 - Take the uniform distribution on $\mathcal{M}$, so every message has $\Pr[M = \cdot] = \frac{1}{|\mathcal{M}|} > 0$
 - Fix any $m' \in \mathcal{M}$ and let $c = \text{Enc}_k(m')$ for $k \leftarrow \text{Gen}$
 - * Then $\Pr[C = c] > 0$, as the definition requires
 - Consider the set $M(c) = \{\text{Dec}_k(c)\}_{k \in \mathcal{K}}$
 - * By *correctness* ($\text{Dec}_k(\text{Enc}_k(m)) = m$ for every k, m), these are the only possible messages that could yield the ciphertext c
 - $|M(c)| \leq |\mathcal{K}| < |\mathcal{M}|$, so there is some m that is not in $M(c)$
 - * $\Pr[M = m \mid C = c] = 0 \neq \frac{1}{|\mathcal{M}|} = \Pr[M = m]$

Where do we stand?

- We defined the notion of perfect secrecy
- We proved that the one-time pad achieves it!
- We proved that the one-time pad is optimal!
 - I.e., we cannot improve the key length
- Are we done?
- Do better *by relaxing the definition*
 - But in a meaningful way. . .

Toward computational secrecy

Perfect secrecy

- Requires that *absolutely no information* about the plaintext is leaked, even to eavesdroppers *with unlimited computational power*
 - Has some inherent drawbacks
 - Seems unnecessarily strong

Computational secrecy

- Would be ok if a scheme leaked information *with tiny probability* to eavesdroppers *with bounded computational resources*
- I.e., we can relax perfect secrecy by
 - Allowing security to “fail” with tiny probability
 - Restricting attention to “efficient” attackers

Tiny probability of failure?

- Say security fails with probability 2^{-60}
 - Should we be concerned about this?
 - With probability $> 2^{-60}$, the sender and receiver will both be struck by lightning in the next year. . .
 - Something that occurs with probability $2^{-60}/\text{sec}$ is expected to occur once every 100 billion years

Bounded attackers?

- Consider brute-force search of key space; assume one key can be tested per clock cycle
- Desktop computer $\approx 2^{57}$ keys/year
- Supercomputer $\approx 2^{80}$ keys/year
- Supercomputer since Big Bang $\approx 2^{112}$ keys
 - Restricting attention to attackers who can try 2^{112} keys is fine!
- Modern key space: 2^{128} keys or more. . .

Roadmap

- We will give an alternate (but equivalent) definition of perfect secrecy
 - Using a randomized experiment
- That definition has a natural relaxation

Perfect indistinguishability: the idea

- $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$, message space $\mathcal{M}$
- Informally:
 - Two messages m_0, m_1 ; one is chosen and encrypted (using unknown k) to give $c \leftarrow \text{Enc}_k(m_b)$
 - Adversary A is given c and tries to determine which message was encrypted
 - Π is perfectly indistinguishable if *no* A can guess correctly with probability *any better than* $\frac{1}{2}$

The indistinguishability experiment

- Let $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ have message space $\mathcal{M}$, and let $\mathcal{A}$ be an adversary.
- Define the randomized experiment $\text{PrivK}_{\mathcal{A}, \Pi}$:
 1. $\mathcal{A}$ chooses two messages $m_0, m_1 \in \mathcal{M}$
 2. Sample $k \leftarrow \text{Gen}$ and, independently, a uniform bit $b \leftarrow \{0, 1\}$
 3. Give $c \leftarrow \text{Enc}_k(m_b)$ to $\mathcal{A}$
 4. $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$ for b
- The *challenge ciphertext* c encrypts one of the two chosen messages. $\mathcal{A}$ knows m_0, m_1 , and c , but not b or k
- $\mathcal{A}$ succeeds if $b' = b$; the experiment outputs 1 on success and 0 otherwise

Perfect indistinguishability: the definition

- Easy to succeed with probability $\frac{1}{2}$...
- Π is *perfectly indistinguishable* if for all attackers (algorithms) A , it holds that $\Pr[\text{PrivK}_{A,\Pi} = 1] = \frac{1}{2}$